

# Adaptive Agent Modes v2 - Candidate Report

Weekly research pass + promotion preparation (release 0.2.0)

Status: capability activation remains 14/14 PASS. The second boundary campaign invalidated its own fixture contract; the fixture has since been versioned to `adaptive-contract-evolution-v2`, a Fixture Admission Gate and canary spend rule now guard the whole spend path, and all twelve zero-provider Candidate gates PASS again. Two completed six-call screens are preserved as invalid evidence. No replacement calls, Stable change or promotion are authorized; this is not Stable guidance.

## Executive result

The previous Vanilla-versus-Full sentinel did not show that agentic guidance was useless. Two shared failures came from invalid evaluation contracts: whitespace-only input was hidden behind the phrase "non-empty", and semantically correct documentation was rejected by a literal substring. The only unaffected pair scored 100/100 and was a ceiling. The correct response was to freeze the conclusion, repair evaluation validity and make components independently measurable.

Candidate v2 implements five runtime modes. Runtime Vanilla is advisory/read-only; coding benchmarks instead use an unscaffolded task-only `vanilla-control`. Selection uses three dimensions: consequence is the only path to Full, while breadth and continuity cap at Mode 3. Planning a critical action is distinct from executing it. The policy chooses the lowest mode covering the evidenced dimensions and escalates on scope growth, ambiguity, repeated verification failure, critical action or session boundary. Numeric ceilings remain hypotheses pending benchmark calibration.

## What changed

### Router precision

The old router selected research rules because an API response contained `source`, security because a schema contained `email`, and migrations because a parser returned an `index`. V2 requires strong phrases or contextual co-occurrence and supports negative matches. All 17 mode cases pass, including the known false-positive and action-intent traps and the decision-time finding-surfacing case.

### Objective integrity and enforcement

The objective compiler covers explicit list items, the task objective and normative prose outside code fences. Every material requirement gets a stable ID and evidence kinds. Ambiguous wording such as non-empty versus non-blank, or semantic versus verbatim documentation, is recorded before implementation.

Completion is fail-closed. Mode 1 receives the minimal Core plus a lossless compact ledger. Mode 2 and above require every applicable ledger row to have reproducible evidence, material ambiguities to be authoritatively resolved, verification commands to pass and protected inputs to retain their hashes. Mode 3 and Full additionally require all six impact-map sections, an adversarial threat model and a re-executable challenge. Wide or multi-session work must leave an evidence-backed checkpoint and actionable handoff. Full also needs human scope approval and an independent verifier PASS. Hidden

graders remain isolated, host-only evaluation evidence and cannot be used as an agent's completion gate.

## **Adaptive cost and model policy**

Modes select durable capability profiles, not product names. The current weekly provider catalog resolves the profile and records the actual model after execution. Expiration blocks automatic selection and creates a model-refresh Candidate; it does not invent an ID or silently alter the user's default.

The first scaffolding screen uses one validated open-world parser fixture and the same resolved model/effort for every arm. It costs six provider calls, not five: five solutions plus the independent Full verifier. There are no replacements, automatic follow-ups or paid calls without a separate approval.

## **Evidence**

- Candidate unit tests: 16/16, plus 8/8 fixture-admission tests.
- Candidate readiness gates after the fixture repair: 12/12 PASS (including the new fixture-admission gate and its test suite); additional provider calls: 0.
- Capability Activation Gate: 14/14 declared capabilities active, with a behavioral probe rather than label-only evidence.
- Mode boundary fixture v2: deterministic controls score Mode 1 proxy 50, reject-on-miss control 72, Mode 2 proxy 78 and Mode 3 reference 100; all pass public tests and only the reference passes the hidden grader. The admission gate's hostile battery confirms per-dimension grading under import bombs, first-call exceptions and wrong return types.
- Router mode cases: 17/17 (including breadth-cap, consequence-beats-breadth and action-intent regressions: wide mechanical work stays at Mode 3, an executable one-line credential rotation selects Full, while explanation and runbook-only requests do not; plus the decision-time surfacing case proving a benchmark-design prompt surfaces the scaffolding-cost finding).
- Research utilization: claims-rules map 42 rules / 23 claims / 55 links with 11 honestly unsupported rules; validator, hygiene scan and 15 research-utilization tests wired as suite gates (see "Research utilization" section).
- Real-world fixture validity: 6/6; every reference scores 100.
- Negative controls: 12/12 pass public tests and are rejected by hidden graders.
- Semantic documentation variants: both formerly rejected agent solutions now score 100 under the versioned semantic oracle.
- Adaptive workspace preflight: 5/5 arms prepared, zero hidden paths, zero provider calls.
- Historical classification: execution integrity PASS; affected quality checks UNINTERPRETABLE; original records unchanged.

## First outcome screen

Campaign `20260714-111231-adaptive-mode-ablation` executed exactly six sequential Codex calls on `gpt-5.6-sol` at medium effort: five solution arms and one independent Full verifier. There were no replacements or automatic follow-ups. Subscription CLI monetary cost was not reported.

| Arm                | Product score | Wall seconds | Observed tokens | Validity                                     |
|--------------------|---------------|--------------|-----------------|----------------------------------------------|
| Vanilla control    | 25            | 26.6         | 38,259          | Invalid control prompt                       |
| Mode 1 Lean        | 100           | 187.7        | 388,416         | Valid but ceiling                            |
| Mode 2 Routed      | 100           | 225.8        | 419,491         | Valid but ceiling                            |
| Mode 3 Assured     | 100           | 217.9        | 421,147         | Valid but ceiling                            |
| Full plus verifier | 100           | 326.1        | 576,757         | Product and verifier PASS; host gate invalid |

The screen cannot support a mode recommendation. Runtime Vanilla had recently become advisory/read-only, but the benchmark mistakenly reused that preset and explicitly prohibited the supposed coding control from editing; it changed no files. Full produced a 100-point solution and its independent verifier passed, but the Windows host finalizer treated the verifier's Linux `python3` launcher as a product failure (exit 9009). Mode 1 through Mode 3 all reached a 100-point ceiling, so they do not discriminate adjacent boundaries.

The historical records remain unchanged. The derivative summary is `candidate-screen-invalid`, its raw Mode 1 recommendation is suppressed, and Stable remains untouched. Candidate now prepares Vanilla as a distinct unscaffolded task-only control and resolves a leading `python3` command to the pinned host interpreter when necessary. Both repairs pass zero-provider regression tests.

## Capability activation audit and boundary fixture

The activation audit now runs a behavioral probe. Every scaffolded arm receives `core.md`; Mode 1 uses a smaller schema-valid ledger with identical requirement IDs; Mode 3+ compiles and gates impact, adversarial, checkpoint and handoff state; and each graded run writes observable-only context telemetry. Missing or pending artifacts make completion fail. All 14 declared capabilities are active and the activation gate passes.

The new `adaptive-contract-evolution` fixture combines open-world Unicode data, strict validation, compatibility, two consumers, copy isolation, observability and semantic documentation. Its deterministic controls produce an ordered 50/78/100 separation while all three pass the deliberately weak public test. The hidden grader is host-only, every check has a public-contract anchor, the reference scores 100 and both flawed controls are rejected. These are capability proxies, not a claim

that model arms will reproduce the exact scores; the next provider campaign remains a separately approved experiment.

## Boundary outcome screen

Campaign `20260714-132536-adaptive-mode-ablation` executed exactly six sequential Codex calls on the pinned `gpt-5.6-sol` medium profile: five solutions and one independent Full verifier. Execution integrity passed with six unique run IDs, no replacements and no automatic follow-up.

| Arm             | Product score | Wall seconds | Observed tokens | Completion gate          |
|-----------------|---------------|--------------|-----------------|--------------------------|
| Vanilla control | 9             | 122.4        | 181,697         | N/A                      |
| Mode 1 Lean     | 9             | 239.3        | 421,357         | PASS                     |
| Mode 2 Routed   | 9             | 256.7        | 526,234         | PASS                     |
| Mode 3 Assured  | 9             | 260.0        | 498,624         | PASS                     |
| Full primary    | 9             | 325.8        | 610,413         | Invalid Windows boundary |
| Full verifier   | -             | 177.8        | 388,076         | Verifier PASS            |

The comparative verdict is INVALID, not a five-way tie. All five solutions and the independent verifier made the same reasonable interpretation: a valid but unregistered entity increments `unknown_entity_type` and raises. The hidden reference silently required pass-through with unchanged attributes, but the public task never states what happens when no registry handler exists. This is `SPEC_AMBIGUITY`.

The grader also wraps the entire check sequence in one exception boundary. The first registry-miss exception replaces the remaining eight dimensions with a single `fixture-runtime` failure, producing identical 9/100 scores even though the rest of each solution was never scored. This is `GRADER_FAILURE`. Full additionally encountered `HARNESS_PLATFORM_MISMATCH`: its independent verifier returned PASS, but Windows saw a discoverable yet broken `python3` alias and final re-execution returned 9009. Candidate now always uses its pinned interpreter on Windows and has a regression for this exact alias condition; historical evidence remains unchanged.

No mode recommendation is permitted. The next fixture version must explicitly define registry-miss semantics, isolate every grader check, include a reasonable reject-on-miss semantic control, and strengthen ambiguity/open-world gates before any new provider approval.

## Boundary campaign postmortem and the admission + canary layer

The invalid boundary campaign consumed 2.63 million observed tokens across six paid calls (five solutions at 181,697 to 610,413 tokens plus a 388,076-token verifier) and produced zero comparative information: every arm scored an identical 9/100. Three `validity_issues` categories explain it.

1. `SPEC_AMBIGUITY`: the task said unseen strings "must work through the runtime registry" while also asking to record `unknown_entity_type` "at the corresponding behavior boundaries". All five solvers and the independent verifier read that as reject-on-miss; only the hidden reference required pass-through. The miss case was never stated.
2. `GRADER_FAILURE`: one exception boundary wrapped all nine hidden dimensions, so the first registry-miss exception collapsed the remaining eight checks into a single `fixture-runtime` row (identical 9/100 floors, plus cp1252 mojibake such as "Å¼Ã³Å,w-æ-°" in the captured diagnostics).
3. `HARNESS_PLATFORM_MISMATCH`: `fixture-contract.json` hardcoded `python3`, which resolves to the broken MS-Store alias on Windows (exit 9009), so the Full arm's completion gate was invalid even though its verifier passed.

The root cause was structural, not one bad fixture: validity machinery existed (`eval_validity.py` covers the six contracted fixtures), but NEW ad-hoc campaign fixtures entered spend through an ungated path -- the preflight checked only workspace mechanics and call counting. The structural fix is a generator-level gate plus a spend-shape rule:

- **Fixture Admission Gate** (`implementation/fixture_admission.py`): a fail-closed CLI producing an admission record with per-check results. It requires the hidden reference at its declared score; a second, independently authored reference at `hidden/reference-alt/` (writing it from the task text alone reproduced the paid failure at zero cost: the alternative interpretation scored 50/100, proving the ambiguity before any spend); negative controls failing hidden grading inside declared score bands while passing public tests; a hostile-candidate battery (import bomb, first-call exception, wrong return types) in which the grader must still report every declared dimension and never score at or above the reference; a host-portability lint over contract/harness files; and interpretation coverage -- an explicit miss-case behavior sentence whenever the task mentions registry/lookup/open-world concepts, plus a `decision_points` section pinning every point where two defensible behaviors exist to the task sentence that resolves it. Preflight and campaign creation refuse to PASS unless a fresh admission record (same fixture, verdict PASS, regenerated after the newest fixture change) exists; the interpretation-coverage check is also applied to the six contracted fixtures through `eval_validity.py`, and all six pass it.
- **Canary spend rule** (data in `ablation-design.json`, enforced by `new_ablation_campaign.py` / `approve_ablation_campaign.py`): a fixture with no prior paid run whose run-record shows `outcome_valid=true` gets a first approved stage of exactly one call on the cheapest arm. The remaining arms require a separate approval that is only constructible from a canary run-record with `outcome_valid=true`, at least two distinct hidden dimensions reported (no floor/ceiling collapse) and every declared grader dimension present. Had this layer existed, the invalid campaign would have cost one call instead of six.

The repaired `adaptive-contract-evolution-v2` fixture pins all five decision points in `task.md`, uses a `{python}` placeholder resolved to the pinned host interpreter, isolates every grader check, and adds a reject-on-miss semantic control; its deterministic controls now separate at 50/72/78/100 and the admission gate returns PASS. All subprocess captures in the runners force `encoding="utf-8"`, `errors="replace"` and every CLI reconfigures stdout, removing the mojibake class entirely.

## Canary finding on v2 and the harder v3 fixture

The one-call canary on `adaptive-contract-evolution-v2` (run-record `20260714-164755236-  
adaptive-contract-evolution-v2-vanilla-t1`, 188k observed tokens) came back `outcome_valid=true` with the **UNSCAFFOLDED vanilla-control arm scoring 100/100**. That is the canary doing its job: it revealed at one call, not six, that v2 is valid but too easy -- a cold pass ceilings it, so scaffolded arms have zero headroom to demonstrate lift. Spend on the remaining v2 arms would have been uninformative.

The successor `adaptive-contract-evolution-v3` (`implementation/mode-boundary-fixture-v3/`, admission verdict PASS) keeps a solvable happy path but buries three failure modes that scaffolding specifically prevents, so the separation lives entirely in dimensions a single cold pass reliably misses:

1. **Omission-prone ( `degraded-path` )** -- a registered handler may raise on some inputs; the contract requires containing it (increment `handler_error` once, fall back to the deep-copied original attributes, do not reject or propagate). This lives once in prose on a rare error path. Objective-ledger scaffolding forces the requirement onto the coverage list; a cold pass lets the handler exception propagate.
2. **Downstream-propagation ( `exporter-propagation` )** -- the audit-export consumer `exporter.export_digest` reaches normalization two hops away through `serializer.serialize`, and must adopt the new registry-aware, `schema_version`-carrying contract. The two obvious consumers ( `render_primary` / `render_audit` ) are named in the bullet list; this indirect one is mentioned only in prose. Impact-map scaffolding enumerates call sites; a cold pass updates the obvious two and leaves the exporter on the legacy path.
3. **Adversarial open-world ( `exact-type-match` )** -- a type with a trailing space ( `"customer "` ) must be an unresolved type, not a trimmed match to a handler registered under `"customer"`; the blank check may use `strip()` but the lookup and the emitted `entity_type` use the original string. Adversarial-verification catches the natural "tidy the key" instinct that collapses the distinction.

The fixture ships a distinguished `plausible-cold-pass` negative control: a clean, correct-looking single-pass solution that implements the happy path and every bullet-list requirement, passes the public test, isolates copies, instruments telemetry -- and omits exactly those three dimensions. It scores **73/100** (failing `degraded-path`, `exporter-propagation`, `exact-type-match`; passing the other eight), sitting in the declared 66-80 headroom band, strictly between the weak `mode-1-local` control (16/100) and both references (100/100). It is the fixture's cheap deterministic proxy for the expected unscaffolded/vanilla failure mode, and

`tests/test_fixture_admission.py::test_v3_plausible_cold_pass_scores_below_reference` locks the design in. Both the primary reference and an independently authored `reference-alt` score 100, and the hostile-candidate battery confirms the grader degrades per-dimension.

## Live v3 canary finding and the v4 fixture

The one-call canary on `adaptive-contract-evolution-v3` ran live on `gpt-5.6-sol` (medium) as the UNSCAFFOLDED vanilla-control arm (run-record `20260714-174322702-  
adaptive-contract-evolution-`



v3-vanilla-t1, outcome\_valid=true, all 11 dimensions reported). It scored **91/100 and missed exactly one dimension: exporter-propagation** -- the indirectly reached audit-export consumer (exporter.export\_digest -> serializer.serialize, two hops away, named only in prose). It **passed** degraded-path (the buried rare-error-path requirement) and exact-type-match (the adversarial .strip() -for-lookup trap). The signal is sharp: this model reads carefully enough to catch stated-but-buried single requirements and common open-world edges, but its reliable blind spot is **multi-hop / indirect downstream propagation** -- impact-analysis territory. v3's headroom was real but too narrow to spend a five-arm main stage on: 9 points, one dimension, n=1, so a muddy result was likely. (v3 now has valid paid history, so its campaign correctly escalates past the canary to the main stage; the canary-forced-without-paid-history invariant is now exercised on v4.)

adaptive-contract-evolution-v4 (implementation/mode-boundary-fixture-v4/, admission verdict PASS) aims the difficulty squarely at that demonstrated weakness and widens it from one dimension to three independent ones worth 35 of 100 points:

1. **Chain A -- exporter-propagation (12 pts).** The audit-export consumer exporter.export\_digest(records, registry) reaches normalization two hops away through serializer.serialize, and must adopt the registry-aware, schema\_version-carrying contract. Like v3's exporter, it is named only in prose, not in the bullet list.
2. **Chain B -- snapshot-propagation (12 pts).** A genuinely separate cache-snapshot consumer snapshot.build\_snapshot(events, registry) reaches normalization two hops away through a **different** intermediary, digest\_helper.digest\_line, in its own module with its own prose sentence and its own output format (v3|type|id|[keys] vs the exporter's 3:type:id:[keys]). Catching Chain A does not reveal Chain B: they share no code and are migrated independently. An impact map enumerates both call sites; a cold pass optimizing for the visible bullets plus the public test updates the obvious render\_primary / render\_audit and leaves both indirect paths on the legacy parse\_record contract.
3. **Hard omission -- repeat-dedup (11 pts).** A state interaction across a re-run: because ingestion is retried, the unknown-type telemetry is per identity, not per call. The first unresolved (entity\_type, entity\_id) sighting counts unknown\_entity\_type; a replayed sighting of the same unresolved pair counts unknown\_repeat and never unknown\_entity\_type a second time. This is a compound condition (unresolved AND already-seen) that only becomes visible when you trace what happens when the same event is re-ingested; it is stated in prose and not unit-tested publicly. A cold pass counts unknown\_entity\_type on every sighting and never emits the repeat metric.

The ten remaining dimensions -- including the now-solvable degraded-path (handler-error containment) and exact-type-match (the trailing-space adversarial edge), both of which the live vanilla arm caught on v3 -- are kept straightforwardly solvable so they are fair rather than headroom. The distinguished plausible-cold-pass control models the live vanilla arm precisely: it implements the happy path, every bullet, the buried degraded path and the adversarial edge, isolates copies, instruments telemetry, matches types exactly, and passes the public test -- and omits exactly the two propagation chains and the re-run dedup. It scores **65/100** (failing exporter-propagation, snapshot-propagation, repeat-dedup; passing the other ten), inside the declared 55-68 middle-low band, strictly between the weak mode-1-local control (11/100) and both 100/100 references.



`tests/test_fixture_admission.py::test_v4_plausible_cold_pass_scores_below_reference` locks the score, band and the exact set of missed dimensions. A new one-call canary is constructible for v4 (`new_ablation_campaign.py --fixture adaptive-contract-evolution-v4 -> stage canary`), one vanilla call, zero provider calls, (`awaiting-explicit-approval`); it has not been approved or run. This wide, three-dimension, all-scaffolding-catchable gap is what makes an n=1 main-stage ordering meaningful where v3's single-dimension gap did not.

## Semantic grading + process metrics

**The live main stage was invalid, caught in flight.** The approved six-call v4 main stage ran five arms on `gpt-5.6-sol` (medium) -- `vanilla`, `mode-1-lean`, `mode-2-routed`, `full`, plus the v4 canary `vanilla --` and every executed arm scored an **identical 70/100** (the sixth, `mode-3-assured`, was halted by the kill switch). Five different scaffolding levels producing the exact same score is not a null result, it is a broken instrument. Root cause, caught live: the hidden grader hard-compared **exact serialization strings** for three dimensions (`consumer-primary`, `exporter-propagation`, `snapshot-propagation`) against arbitrary formats the task never pins. The grader wanted render `customer:2`, exporter `3:customer:1:['n']`, and snapshot `v3|account|7|['m']` -- **three mutually inconsistent conventions** (schema-prefix here, schema-suffix implied there, `v3|` pipe-form elsewhere). The `full` arm emitted `customer:1:['n']:3`; `mode-1-lean` emitted `customer:1:v3:['n']`. Both carried `schema_version` and applied the registry **correctly in substance** on both indirect chains -- they simply chose a different delimiter/placement from the grader and from each other. The arms did the work; the grader measured formatting.

**The fix: grade content, not strings.** `hidden/grade.py` now parses each serialized line tolerantly (token extraction + a standalone-version regex) and asserts the required *facts*: the line goes through `normalize_event` (the registry handler-added key is present), carries the `schema_version` value 3 in any format (`3:`, `:3`, `v3|`, `schema=3` ...), and encodes the correct `entity_type`, `entity_id` and attribute keys. It still rejects a line that omits the version or leaves the legacy `parse_record` path. The exporter probe is representative:

```
digest = exporter.export_digest(events, registry)
ok, missing = _line_ok(digest[0], "customer", "1", ["n"]) # resolved: handler adds key 'n'
if not ok:
    return False, (f"audit line for the resolved event is missing {missing} "
                  f"(registry not applied or schema_version not carried): got {digest[0]!r}")
```

where `_line_ok` requires the `entity_type/entity_id/key` tokens plus `_carries_version(line, 3)` (`re.search(r"(?!<\d)3(?!>\d)", line)`). Per-check `record()` isolation and the `BaseException` boundary are unchanged; the reference still scores 100, and a different-but-valid format now also scores 100.

**The semantic re-grade** (`evidence/main-v4-semantic-regrade.json`, **zero provider calls**). Each saved `workspace/src` was copied into a fresh sandbox and re-graded under the semantic grader plus `process_metrics.py`:

| arm              | exact | semantic | tokens    | consumer-enum | consumer-edit | self-attestation gap |
|------------------|-------|----------|-----------|---------------|---------------|----------------------|
| full             | 70    | 100      | 1,626,616 | 1.0           | 1.0           | False (trustworthy)  |
| mode-1-lean      | 70    | 100      | 658,269   | N/A           | 1.0           | N/A                  |
| mode-2-routed    | 70    | 100      | 579,344   | N/A           | 1.0           | N/A                  |
| vanilla (main)   | 70    | 100      | 313,237   | N/A           | 1.0           | N/A                  |
| vanilla (canary) | 70    | 100      | 212,369   | N/A           | 1.0           | N/A                  |

**What it reveals about scaffolding -- honestly: on this task, neither the outcome nor process quality separated the arms; only token cost did.** Once format noise is removed, all five arms are semantically identical at 100 -- every arm, including bare vanilla, already routed both indirect chains through `normalize_event`, applied the registry and carried the version. Scaffolding did **not** lift the outcome here because the model was already capable of the multi-hop propagation this fixture targets (a real finding: the v4 headroom hypothesis, that indirect propagation is a reliable blind spot, did not hold for this model at this difficulty). The scaffolding's remaining footprint is **process artifacts** (only `full` produced an `impact-map.json`, enumerating 5/5 ground-truth consumers) and **token cost** (`full` spent ~5.2x the main vanilla and ~7.7x the canary vanilla for the same graded outcome). Crucially, the key instrument -- `self_attestation_gap` -- is **False** for `full`: it marked its impact-map consumers "verified", and under semantic grading its consumer dimensions all pass, so that "verified" claim was **truthful in substance**. The exact grader was wrong, not the agent. (The gap instrument does fire correctly when a run claims "verified" but is semantically wrong on a consumer dimension; `tests/test_fixture_admission.py` locks both directions.)

**The admission-gate fix that closes the hole ( `format-adversary` ).** The v4 grader passed admission originally because its own `reference-alt` happened to use the *same* format as `reference`, so nothing ever exercised format-agnosticism. The gate now requires, for any fixture that declares `serialized_output_dimensions`, a `hidden/reference-format-alt/` that is semantically correct but uses **deliberately different** delimiters/placement (render `entity[customer]#2`, exporter `customer@1` keys=[n] schema=3, snapshot `[3] account → 7 :: m`), and it fails admission if the grader scores that adversary below 100 or if the adversary is a verbatim copy of the reference:

```
outcome = self.grade_overlay([overlay])
grader_agnostic = (outcome["public_pass"] and outcome["hidden_json"]
                  and outcome["score"] == self.expected_score and not missing)
passed = grader_agnostic and bool(differing) # differing = format-bearing files that differ
```

Pointed at the **original exact-string grader**, this check reproduces the failure exactly: the format-alt scores 70 and admission FAILs with a `format-brittle grader` message -- i.e. the fix would have rejected the very grader that invalidated the main stage, before any spend. Pointed at the semantic grader it scores 100 and PASSes. The check is wired for every fixture declaring

`serialized_output_dimensions` (v4 today) and skips others with a recorded reason; the v4 admission record now carries all 7 checks at PASS. Control bands are unchanged because the controls fail on *substance*: `plausible-cold-pass` still scores 65 (it leaves both chains on the legacy path and carries no version there), `mode-1-local` 11, references 100. `run_candidate_evals.py` gates the re-grade (`semantic-regrade-v4`) at 20/20, zero provider calls.

## | Spec-first planning layer

Two real bodies of evidence ground this layer. In the owner's HermesAirflow repo, tasks like "build the DC WIRE product" and "adapt the NY Medicaid state adapter" cost many human iterations because correctness depended on implicit house conventions discoverable only by exploring the code (a load-bearing 26-column row contract, a required shell\_resolver-before-resolve ordering that degrades silently, a STOCK-vs-FLOW double-counting trap documented only in a docstring, point-in-time date discipline, ambiguous-brand-token guards) -- the task text contained almost none of the real requirements. The live v4 canary showed the same class quantitatively: the unscaffolded model scored 70/100 missing exactly the indirect multi-hop consumers. And a 22-defect postmortem of the finished DC WIRE product traces its failures to recurring classes (silent zero-row loads at transport success, untested verifiers used as confidence arbiters, parsers built for the rarest format, mid-flight domain-invariant discoveries, cold-start failures, guessed ambiguous values, double-writers). Spec-first exists because those defects were all discoverable BEFORE implementation by structured exploration.

**Clarity, the fourth selection axis** (`modes.json` `axes.clarity`, `adaptive_router.specification_clarity`). A task is `underspecified` when it implies a real scope ("new product/source/adaptor/integration", "like the existing X", "consistent with how the existing sources work") while pinning almost none of its own acceptance criteria (no `exactly` / `must not` / `raises` sentences, low compiled requirement count, or material ambiguity/breadth phrases). Underspecified mutating work floors at mode-2 and receives the `spec-synthesis` capability on top of the selected mode; clarity never touches the human gate and never selects full. The classifier is deliberately biased toward precision: `underspecified` REQUIRES an implied-scope phrase, so mechanical, read-only, doc-only and explicitly pinned tasks never buy the planning phase. All pre-existing mode cases route unchanged (the suite is now 17 with the decision-surfacing case); the four Hermes-style prompts classify underspecified and the v4 fixture task, being fully pinned, classifies well-specified (`tests/test_spec_first.py`).

**Spec synthesis** (`implementation/spec_synthesis.py`, dropped into `.agentic/` by `prepare_context.py`). `compile` scaffolds `spec.json` / `spec.md`; the agent must fill: an objective restatement; a **surface inventory** enumerated from the code with free-form, open-world `kind s` (external I/O, persistence, state/concurrency, parsers, contracts are a starting lens, never a schema); a **convention inventory** citing real files (existence-checked); **decision points** pinned by evidence or explicitly escalated to the owner (which blocks); a **risk register** where every risk needs a mitigation plus a runnable verification and must trace to a surface or requirement, and every surface needs a risk or a reasoned not-applicable; **rejected approaches** (so reverted over-engineering is not redone); **acceptance tests** as self-verifying commands; a **coverage argument** (the inventory carries its own confidence claim and is never "done"); and the **frozen ledger** -- the union of task-text requirements

(from `objective_compiler`, undroppable) and spec-discovered requirements. Validation is generative and adversarial, not enumerative: `validate` checks traceability rather than coverage-of-a-list, and additionally requires a **separate adversarial pass** over task+workspace+spec hunting for (a) unpinned decision points, (b) surfaces missing from the inventory, (c) risks without a real verification. The 22-defect postmortem taxonomy is demoted to seed examples in `modules/risk-discovery.md`, which states explicitly that novel classes are expected and the list must never be treated as a rubric -- the "discovery sample as production allowlist" anti-pattern is exactly what open-world policy forbids.

**Ledger freeze -- anti-scope-shrink** (`pre_submit_gate.py`). The first PASSing `validate --freeze` records `spec_sha256` over the canonicalized frozen ledger + acceptance tests. At completion the gate re-validates the spec, recomputes the hash, and on divergence applies the row-level rule: ANY removal, rewrite or downgrade (`not_applicable` / `rejected` in spec or evidence) of a frozen requirement without an `owner_scope_changes` entry (human approval string + reason) fails; additions are always allowed. Every acceptance test is re-executed by the gate and must exit 0. Real outputs: honest full-scope completion -> `verdict PASS, completion_allowed true`; after silently dropping one discovered requirement -> `verdict FAIL: "frozen requirement REQ-DISC-APPENDLOG was removed without an owner_scope_change entry"` (exit 1). `tests/test_spec_first.py` (19 tests) locks all of it: fail-closed spec cases, freeze semantics, gate integration, prepare\_context artifacts.

**The clarity fixture** (`implementation/mode-boundary-fixture-clarity/`, id `implicit-conventions-v1`, admission verdict PASS). A miniature DC WIRE in a fresh mini-domain: a working market-events ingestion library (alpha/beta feeds -> shared product table + change log) with five implicit conventions burying correctness, plus the deliberately SHORT task "Add a new source `gamma` ... keep everything consistent with how the existing sources work". The conventions: a load-bearing positional column contract (`rows.COLUMNS`); a scrub-BEFORE-resolve ordering whose violation silently resolves ambiguous prose tokens into issuers; an append-only change-log discipline documented only in a docstring (amend = new restate row, never an update); a point-in-time rule (`as_of` = published date, never effective -- the look-ahead trap); and a loud-failure rule for error-shaped/zero-row payloads at transport success (the postmortem's silent-failure class). Because the task is underspecified by design, the admission gate's interpretation-coverage check is class-aware: for `fixture_class: underspecified-by-design` it instead verifies **convention anchors** -- every hidden-grader dimension must be traceable to a quoted, existence-checked convention in the public workspace (the spec-discovery equivalent of public anchors). The `plausible-cold-pass` control (a clean solution that passes the public tests and reuses the shared row builder but never goes looking for conventions) scores **47/100**, failing exactly `order-dependency`, `point-in-time`, `changelog-discipline`, `silent-zero-rows`; the weak control scores 18, both references 100. That 53-point, four-dimension gap is the headroom claim for a future paid canary; a one-call canary campaign is constructible (stage `canary`, 0 provider calls, `awaiting-explicit-approval`) and has not been approved or run. This fixture also inherits the unpaid-fixture role in `campaign_safety_test.py` now that v4's live canary gave it paid history.

**Honest limits.** Spec synthesis structures discovery; it cannot force a model to find every convention -- an agent can fill a syntactically perfect spec whose convention inventory still misses the trap (the fixture measures exactly this residual). The freeze is enforced against the recorded `spec-freeze.json`; an

agent that regenerates the freeze after shrinking discovered rows defeats the row-level check locally (task-text rows remain undroppable because validate recompiles them from the task), so the freeze file belongs in the protected/audited set at promotion time. Discovered frozen requirements are enforced through freeze integrity and acceptance-test re-execution, not through per-requirement evidence rows. The clarity classifier can misfire both ways: a false positive wastes a planning phase on a well-specified task (bounded cost: one spec + validation), a false negative degrades to today's behavior (unbounded cost on the wrong task: the Hermes iteration tax); despite that asymmetry we chose the precision bias because a completion-blocking gate that fires wrongly erodes trust in every other gate, and the phrase list is cheap to extend as false negatives are observed and added to the test matrix.

## Research utilization: claims-rules traceability, decision-time surfacing, vault hygiene

**The concrete miss this layer exists to fix.** `Research/knowledge-base/2026-07-12-findings.md` already contained, cited and dated 2026-07-12: "the full multi-agent harness cost ~20x a solo agent" and "periodically remove harness components since each encodes an expiring assumption" (line 30), and "naive self-eval 'confidently praises mediocre work'" (line 20). Those findings EXACTLY predicted this week's benchmark results - scaffolding was pure token overhead on the in-context v4 task ( `full` spent ~5.2x the main vanilla for the same semantic 100), and the self-attestation instrument became the full arm's key process signal. Nobody surfaced them when the mode ladder and the benchmark program were designed: nine provider calls were spent rediscovering what the corpus already said. The research pipeline was write-only. Three structural gaps explain it: no mechanism routed research claims into design decisions; rules were not linked to the claims justifying them (REQ-RES-004 pending), so claim expiry could never flag dependent rules; and nothing kept the corpus and its surrounding knowledge surfaces honest (REQ-RES-005 pending) - the earlier docs review fixed contradictions and orphans by hand exactly once.

**Mechanism 1 - claims->rules traceability** ( `implementation/claims_ledger.py` + `claims-rules-map.json` ). A machine-readable map linking 42 material rules (all ten Core runtime items, every stable knowledge module and policy, the candidate modules, twelve modes.json capabilities/thresholds, the router precision/budget rules, five benchmark-protocol rules) to 23 harvested claims (13 knowledge-base findings, 4 registry primary sources, 6 locally verified candidate claims) through 55 rule->claim links, each claim carrying source URL, confidence and a recheck/expiry date. The validator FAILs the suite on structural problems (unknown claim references, missing artifacts, malformed claims), WARNs on expired claims and flags their dependent rules, and reports the unsupported count. The honest result: **11 of 42 rules have no research backing** - `RULE-MODE-NUMERIC-CEILINGS` (self-declared uncalibrated priors), `RULE-MODE-BREADTH-CAP` and `RULE-MODE-DOWNGRADE-POLICY` (parts of the mode ladder are design decisions, not research), `RULE-MODE-CLARITY-SPEC-SYNTHESIS` and `RULE-CKM-RISK-DISCOVERY` (grounded only in local Hermes/postmortem evidence), `RULE-BENCH-CANARY` (born from our own two invalid paid campaigns, no external prescription), `RULE-CORE-04` , `RULE-CORE-10` , `RULE-POL-GENERALIZATION` , `RULE-KM-DATABASE-MIGRATION` , `RULE-CKM-RELIABILITY-CONTRACT` . Each records `local_evidence` stating what actually motivated it; the validator rejects an unsupported rule that does not.



**Mechanism 2 - decision-time research surfacing** ( `adaptive_router.py` , `detect_decision_type` + `surface_relevant_findings` , `findings-index.json` ). When a task classifies as a design/benchmark/architecture decision (phrase-anchored: "design the", "should we add", "benchmark", "compare modes", "architecture", "choose between", "ablation"...), the routing result and the Context Pack ( `relevant-findings.json` in `.agentic/` ) carry the top topic-matched findings from a 21-entry index built over the knowledge base, each with a one-line claim, tags and source URLs. Detection is deliberately precision-biased: implementation wording like the fixture tasks' "must adopt the new contract" is execution, not a decision, and gets nothing - benchmark arm contexts stay uncontaminated. Proof in the mode-case suite (case `benchmark-design` , 17/17): the prompt "Design the benchmark protocol for comparing modes of the adaptive harness: should we add a multi-agent verifier arm..." surfaces `F-2026-07-12-021` (the ~20x scaffolding-cost / strip-scaffolding finding) ranked first with five matched terms, followed by `F-2026-07-12-011` (naive self-eval praises mediocre work). This is the exact retrieval that would have put the 20x finding in front of the mode-ladder design.

**Mechanism 3 - vault hygiene loop** ( `implementation/vault_hygiene.py` , zero provider calls, REPORT-ONLY). The governance-adapted vault-maintenance kernel: a periodic scan over Core/, both router catalogs, the knowledge base, the source registry, workstreams/, Models/ and the candidate docs, reporting (a) cross-reference integrity including tools no doc mentions, (b) staleness (catalog expiry, source rechecks, active-workstream age, claim expiry), (c) contradiction CANDIDATES via targeted heuristics only - conflicting requirement statuses across ledgers, v1-workflow assertions where v2 files exist, diverging package statuses - explicitly labeled for human review, and (d) orphans (uncataloged modules, findings no rule consumes, workstream INDEX drift). It NEVER modifies anything; output is a dated JSON+Markdown report under `evidence/` , and the suite gate requires the report generated plus zero broken cross-references inside the candidate package itself. A candidate skill file ( `implementation/skills/weekly-hygiene.md` ) specifies the future promoted `/weekly-hygiene` command with the never-delete, review-before-commit contract.

**First hygiene report headline (2026-07-15, informational).** Broken cross-references: 0 (package: 0). Tools no scanned doc mentions: 5 ( `handoff-check.ps1` , `init.ps1` , `loop-checkpoint.ps1` , `model-route.ps1` , `rollback-release.ps1` ). Stale items: 0 today - the model catalog and source registry both roll over 2026-07-19, so the first real staleness findings are four days out (the far-date detector self-test fires on both). Contradiction candidates: 3 - a bootstrap workstream still asserting v1 research-workflow artifacts without a legacy marker (twice), and this package's own `run-manifest.json` status ( `boundary-campaign-invalid-fixture-repair-required` ) lagging `handoff.json` ( `fixture-v2-admitted-canary-gated ...` ) by several lifecycle stages. Orphans: 10 findings no rule consumes - the eight Codex configuration findings (platform reference, fairly rule-less) plus `F-2026-07-12-014` (3-5 agent orchestration cap) and `F-2026-07-12-018` (clean-context exploration sub-agents), which are genuine gaps: the corpus contains orchestration guidance no current rule uses.

**Live drift caught during this work.** At baseline the suite was 18/20: the clarity fixture's one-call canary had run live on 2026-07-15 morning, giving `implicit-conventions-v1` paid history and silently breaking the two gates that assumed it was the unpaid fixture exercising the canary rule. That is precisely the ungated-drift class this layer targets. The structural fix (not a test edit): `scan_paid_history` and `new_ablation_campaign.py` accept a `--runs-dir` override so lifecycle

tests synthesize the no-paid-history state deterministically instead of depending on mutable

`Evals/runs` contents.

**Honest limits.** Contradiction detection is heuristic grep over declared surfaces - it finds status-field conflicts and v1-phrase drift, not semantic contradictions in prose (a stale "12/12 gates" count in an old report section survives it). The claims harvest is a first pass by one agent: link quality and tag quality are unreviewed, some "supported" links are one-source practitioner claims (confidences recorded per claim), and the 20-40-link target was exceeded (55) because the honest map wanted more edges, not fewer. Topic matching in the findings index is keyword overlap, not semantics: a decision phrased without any indexed term surfaces nothing, and the phrase-anchored decision detector will miss decisions phrased as plain questions ("is Full worth it?" only fires via "worth the complexity"/"is it worth adding" style anchors). All three mechanisms are candidate-only until promoted.

## | The scale fixture: the remaining untested axis

**Four fixtures in a row could not separate any arm from 100 under fair grading.** The pattern: v2's canary vanilla scored **100** outright (run `20260714-164755236`); v4's five live arms all re-graded to **100** under the semantic grader (the exact grader's identical 70s were pure format noise, `evidence/main-v4-semantic-regrade.json`); the clarity fixture's live vanilla canary scored **100** on the first pass (run `20260715-110524196`), discovering all five implicit conventions unaided in 84 seconds and 121k tokens; and v3's lone 91 dissolves under the same lens -- its single failed dimension shows the arm's exporter emitting `customer:1:['n']:2` (evolved contract applied, version carried, keys correct; only the version *placement* differs from the exact grader's expectation), i.e. semantically also 100. Across those four fixtures `gpt-5.6-sol` discovered implicit conventions, routed multi-hop propagation and respected house rules **unaided** -- whenever the codebase fit a single comprehension pass (the largest of them: 12 files, 52KB). The owner's real pain (HermesAirflow DC WIRE: a 26-column load-bearing contract, six DAGs with hidden ordering, STOCK/FLOW traps, a shared layer mid-migration, ~24 scrape-only stub adapters confusing scope) does not fit one pass. Scale is the one axis every fixture so far has failed to test, and it is exactly the axis the scaffolding (spec-first discovery, impact-map enumeration, checkpoints) is supposed to buy correctness on.

**The fixture** (`implementation/mode-boundary-fixture-scale/`, id `implicit-conventions-scale-v1`, `fixture_class: underspecified-by-design`, admission verdict **PASS**). A synthetic market-events platform in the clarity fixture's mini-domain, scaled to defeat one pass: **57 Python modules / ~2.5k Python lines, 72 files / ~3.9k total lines / ~133KB (~33k content tokens)** across `core/` (row contract, append-only change log with a closed kind vocabulary, point-in-time dates, ambiguity guard, state-identity hashing, a shared house flow mid-migration), `reference/` (venue registry, 60-issuer resolution universe with aliases and parent chains, containment guard lists, calendars, symbology, instruments), four live source parsers + a resolution ladder + thin scheduler shells encoding run order, and seven consumers including client-facing exports. The task is one short paragraph: *add source `epsilon` end-to-end, consistent with how existing sources work*. Every one of the 14 hidden dimensions is anchored to a quoted, existence-checked in-repo convention (`convention_anchors`), and the load-bearing ones are deliberately **spread**: the evidence lives  $\geq 2$  modules away from where epsilon's code



must respect it, and two dimensions are only visible by reading two distant files together (the contract's `distance_map` documents every pair).

The signature distant pair is the mid-migration shared layer. `src/core/normalize.py` states the rule:

MIGRATION: gamma and delta are migrated to house\_flow. alpha and beta still carry the same steps inline ... Every NEW source starts on house\_flow; never copy the inline flow out of alpha\_feed or beta\_feed into new code

and the enforcement lives two directories away in `src/consumers/run_health.py`:

```
_LEGACY_SOURCES = frozenset({"alpha", "beta"})
...
if flow.get(source, 0) < loaded[source]:
    flags.append("%s loaded %d rows but recorded %d house_flow passes: the source "
                "bypassed the shared flow (core/normalize.py)" % (...))
```

An agent whose exploration starts at the nearest example -- `alpha_feed.py`, alphabetically first, a complete and *internally correct* parser -- copies the one template the platform says never to copy; nothing errors, only the flow counter stays silently at zero. The same geometry repeats across the other buried dimensions: the closed changelog vocabulary ( `core/changelog.py` ) is enforced by a consumer assertion ( `consumers/audit_trail.py` hard-fails replay on an unknown kind) and alpha's own restate mapping is keyed off a numeric `rev` field epsilon does not have, so it does not transfer by copying; the scheduler run order ( `scheduler/registry.py` + `run_resolution.py` : a shell scheduled after resolution silently vanishes from exposure for a day); and three deliberately silent-skip consumer registries (venue registry -> dashboard, `SOURCE_SECTIONS` -> client export, `VENUE_CHANNELS` -> desk alerts) where a missing epsilon entry produces no error anywhere -- the venue's events just never appear where a human looks.

**Decoys force scope judgment.** Ten scrape-only stub sources (the Hermes `medi_XX` pattern: fetch descriptor, capture dossier, archived-raw index, `load_events` raising `NotImplementedError` ) dilute the sources directory, and a root-level `PLAN.md` overstates the platform -- "Phase 2 complete - all fifteen venues live in production", the half-written `sigma` stub blessed as "the modern template", an instruction to register the full venue roster in the dispatcher, and a `rebuild=True` first-month policy that contradicts the append-only discipline. The truth is discoverable from code ( `src/pipeline.py` : "Only sources with a working parser are registered here; scrape-only stubs stay unregistered until their parser lands.", plus the truthful `RELEASES.md` ). The `scope-judgment` dimension probes it directly:

```
registered = set(pipeline.SOURCES)
extra = sorted(registered - (set(REAL_SOURCES) | {"epsilon"}))
if extra:
    return False, ("scrape-only stubs wired into the pipeline registry: %r. The registry "
                  "docstring scopes registration to sources with a working parser; "
                  "PLAN.md's fifteen-venue roster is aspirational" % (extra,))
```

The **plausible-cold-pass control scores 48/100** (declared band 40-60), missing exactly seven dimensions, each distance-driven: `migration-path` (copied the legacy alpha template; the MIGRATION note and `run_health` enforcement live in files it never opened), `changelog-discipline` (passed epsilon's "correction" category straight through as the kind; the closed vocabulary and its audit enforcement are two modules away and the template's mapping does not transfer), `run-order`, `venue-config`, `consumer-exposure`, `consumer-alerts` (three subtrees -- `scheduler/`, `reference/`, `consumers/` -- the happy path never touches), and `scope-judgment` (followed PLAN.md's roster instruction). Everything the alpha template itself teaches it gets **right**: loud zero-row failures, published-date keying, scrub-before-resolve, the positional row contract, state-identity dedup. The weak `mode-1-local` control scores 16, both references and the format-adversary 100.

`tests/test_fixture_admission.py::test_scale_plausible_cold_pass_scores_below_reference` locks the band and the exact missed set; `test_scale_workspace_exceeds_single_pass_size` locks the scale property itself ( $\geq 40$  modules,  $\geq 100$ KB, the ten stub decoys and PLAN.md's overstatement must survive).

**Admission and harness.** All seven admission checks PASS (dual references at 100, format-adversary at 100 over the contract-declared format-bearing files `exposure_export.py` / `alerts.py` -- a new contract key, since the serialized dimensions here live in per-source consumer registries rather than v4's serializer files -- control bands, hostile battery degrading per-dimension across all 14 checks, platform lint, convention-anchor coverage). The fixture declares `process_ground_truth` (7 consumer/registry files, 7 required edits, 4 consumer dimensions), and `process_metrics.py` computes `consumer_enumeration_completeness` and the `self_attestation_gap` against it (locked by `test_scale_process_metrics_compute_on_synthetic_run`). Campaign creation honors a new `campaign_loop_overrides` contract key: the scale canary is constructed with **40 turns / 50 wall-minutes per call** instead of 18/25 (recorded in the campaign under `loop_overrides`, with the reason), because at the default budget a low vanilla score would measure truncation, not prioritization -- the axis under test is *what the agent chooses to read*, not whether the clock ran out. Overrides do not leak to other fixtures (locked by test). The one-call canary is constructible (`stage: canary`, one vanilla call, 0 provider calls, `awaiting-explicit-approval`) and has **not** been approved or run; the scale fixture also takes over the genuinely-unpaid canary-rule role in the lifecycle tests. Suite: **25/25 PASS, zero provider calls**.

**Honest expectation and residual risk.** The deterministic cold-pass proxy is an *authored model* of single-pass behavior, not a measurement of the live model. The honest case that the live vanilla arm scores below 100 here: the workspace's ~33k content tokens across 72 files cannot be exhaustively read *and* acted on within even the raised 40-turn/50-minute interactive budget without deliberate prioritization (the clarity run spent 121k tokens reading a 52KB workspace one-shot; this one is 2.6x the bytes, 6x the files, with three registries whose absence produces no failing signal anywhere the agent would look, and a salient root-level document actively pointing away from three of the graded dimensions). The equally honest counter-case: 33k tokens still fits the model's context window in principle; a model that greps for `epsilon`, reads every `*_feed.py` header, or simply reads `docs/RUNBOOK.md` (which honestly summarizes onboarding, deliberately -- discoverability must be fair for the fixture to be valid) can reach 100 without scaffolding, exactly as it did four times before.

Confidence the live vanilla arm scores < 100: **moderate at best (roughly a coin flip, tilted slightly toward a miss on at least one of the seven distance dimensions)** -- the four-fixture record is an unbroken streak of the model out-reading our headroom designs, and this design differs in kind (breadth + decoys + silent skips), not just degree. **If the vanilla arm still scores 100 at this scale, that is a decisive null:** scaffolding provides no correctness lift for this model class at any tested size, and the environment's value proposition repositions from correctness amplification to guardrails (spend gates, validity gates, fail-closed completion) plus cost discipline -- which the v4 token data (5.2x cost for identical outcome) already quantifies.

## | The scale canary result and the final verdict (2026-07-15)

The scale canary ran (campaign `20260715-154015`, one call, ~446k observed tokens, 40-turn/50-minute budget): **the unscaffolded vanilla arm scored 100/100 on all 14 dimensions** — including all seven distance-driven conventions, the three silent-failure registries, the wrong-template trap, and scope-judgment (it ignored PLAN.md's aspirational roster, did not wire the decoy stubs, and judged the real scope from code). Eight files changed, one pass.

That closes the question this package set out to answer. Across five fixtures and ten provider calls (v2, v3, v4, `implicit-conventions-v1`, `implicit-conventions-scale-v1`), the unscaffolded model scored 100 under every fair semantic grader, and two independent fixture designers targeting empirically demonstrated weaknesses could not construct a fair in-context task it fails. **The correctness-lift hypothesis for scaffolding is a decisive null for this model class on in-context, single-session tasks** — exactly as the knowledge base's own finding predicted ("strip scaffolding as models improve; each component encodes an expiring assumption").

## | Expected impact — repositioned

The environment's measured value proposition is therefore:

1. **Cost discipline** (the primary quantified win): scaffolded arms spend 5-8x the tokens of vanilla for identical outcomes (v4 semantic re-grade; full = 1.63M vs 212k). Adaptive selection that picks the cheapest sufficient mode is pure savings; correctness is not the trade-off it was assumed to be.
2. **Guardrails:** fail-closed hooks, promotion gates that cannot be self-attested, protected Stable surface, fixture admission + canary spend gates — all live and tested.
3. **Governance-scaffolding, not correctness-scaffolding:** the human gate on critical consequence, durable checkpoints for continuity, and the spec-freeze anti-scope-shrink boundary retain their value as *control and audit* mechanisms — they protect against risks other than model capability (irreversible actions, silent scope narrowing, session loss), and none of the null results touch that rationale.
4. **Research utilization:** claims-rules traceability (RULE-MODE-LADDER now carries this negative outcome as evidence), decision-time findings surfacing, and the vault hygiene loop keep the corpus load-bearing so the next such lesson is read before it is re-purchased.

Untested and deliberately not chased: workspaces exceeding the context window (true Hermes scale) and multi-session continuity. Mode 1 remains the right default for small reversible work; the ladder's remaining differentiators are cost ceilings and governance controls, not quality claims. Promotion scope is now an owner decision under this repositioned framing.

## Promotion prepared (release 0.2.0)

The owner approved promotion under the repositioned framing, and the promotion is now PREPARED (not executed): `promotion/manifest.json` is rebuilt (status `awaiting-approval`, 252 files, fresh stable-manifest hash) over a fully additive payload — the classic router ( `tools/route.ps1` default path + `Router/catalog/modules.json` ) is byte-for-byte behavior-preserved and the adaptive layer lands beside it as `tools/route.ps1 -Adaptive` → `tools/adaptive/` (Python), `Runtime/adaptive-modes.json`, `Router/catalog/adaptive-modules.json`, knowledge modules in `Core/knowledge-modules/`, claims/findings under `Research/`, docs in `Setup/`, boundary fixtures in `Evals/adaptive-fixtures/` (deliberately NOT in the stable fixture catalog), and the `/weekly-hygiene` command content staged at `Claude/commands/` (the live `.claude/commands/` is a forbidden promotion target). The dry-run preview ( `tools/promote-candidate.ps1`, no `-Approve` ) reaches exit 3, and a full post-promotion simulation in a temp repo copy kept knowledge-routing 12/12, fixture-discrimination 19/19, PowerShell/JSON parse clean, and both routing paths working. Caveat for the approval run: `release-gate.ps1 -Mode full` at `-Approve` time is EXPECTED to fail `objective-complete` (open objective requirements unrelated to this increment); the owner performs the approval with the recorded operator skip — no gate is weakened. Fixture repairs for two stable catalog fixtures were deliberately left OUT of the payload (the repaired grader delegates to the stable grader path and would recurse if promoted onto it); they need their own candidate.

## Sources used

| Primary and empirical                     | Platform and safety                               |
|-------------------------------------------|---------------------------------------------------|
| <a href="#">Evaluating AGENTS.md</a>      | <a href="#">Anthropic: context engineering</a>    |
| <a href="#">Probe-and-Refine guidance</a> | <a href="#">Anthropic: long-running harnesses</a> |
| <a href="#">ContextBench</a>              | <a href="#">OpenAI Codex documentation</a>        |
| <a href="#">METR productivity study</a>   | <a href="#">Claude Code documentation</a>         |
| <a href="#">SWE-bench</a>                 | <a href="#">NIST SP 800-218A</a>                  |
| <a href="#">OpenHands Index</a>           | <a href="#">OWASP MCP Security</a>                |

YouTube, podcasts and social/engineering feeds remain research radar inputs in the persistent source registry; they do not override primary evidence or local evals.